

P2P Network Assignment: Viva Guide

CSE 433 - Blockchain & Distributed Security Lab | Python TCP Sockets | Answers to Section 20 questions and core theory

Part 1: Answers to the 20 Viva Questions

1. What is the difference between a client and a server?

A server waits (listens) for incoming connection requests and provides a service. A client initiates a connection to a server. The difference is the **role in a connection**, not the type of machine.

2. Why can one peer act as both a client and a server?

Roles are per connection, not per machine. A peer runs a listening socket (`bind`, `listen`, `accept`) to receive connections and can also create outgoing sockets (`connect`) to reach others. Both run at the same time in different threads, so **every peer = TCP server + TCP client**.

3. What is the purpose of an IP address?

It identifies a host (network interface) so packets can be routed to the correct machine, e.g. 192.168.1.10.

4. What is the purpose of a port number?

It identifies a specific application/service on a host (16-bit number, 0-65535). The IP address gets data to the machine; the port gets it to the right process. IP + port = socket address.

5. Why do different peers need different ports on the same computer?

They all share the same IP (127.0.0.1) and only one process can bind to a given IP:port at a time (otherwise "Address already in use"). The port distinguishes Alice (5000), Bob (5001) and Charlie (5002).

6. What happens when `connect()` is called?

The client asks the OS to establish a TCP connection to the given IP:port. The OS performs the **three-way handshake** (SYN, SYN-ACK, ACK). On success the socket is ready for send/receive. If nobody is listening: *ConnectionRefusedError*; if unreachable: timeout.

7. What happens when `accept()` is called?

It blocks until a client completes a connection on the listening socket, then returns a **new socket** for that client plus the client's address. The original listening socket stays open to accept more connections.

8. Why is TCP used instead of UDP?

TCP is reliable (retransmits lost packets), ordered, error-checked and has flow control. Files must arrive complete and in order or a video/ZIP will be corrupt. UDP is faster but gives no delivery or ordering guarantee.

9. Why are threads used?

`accept()` and `recv()` are **blocking**. Without threads the program would freeze on one call and could not accept new peers, serve other peers, or keep the GUI responsive. Typical design: one accept thread, one receive thread per connection, GUI on the main thread.

10. What is the purpose of the HELLO message?

It is the application-level **handshake**. TCP only gives a connection between IP:port pairs; it does not say *who* the peer is. HELLO exchanges peer ID, name and listening port so each side can identify the other and list it in the connected-peer list. The receiver replies with *hello_ack*.

11. What is message framing?

Defining where one message starts and ends inside a byte stream. Here: [4-byte length][JSON payload]. The receiver reads 4 bytes to get the length N, then reads exactly N bytes.

12. Why can't we assume one `recv()` call gives one complete message?

TCP is a **byte stream**, not a message stream. One `recv()` may return only part of a message (split across packets) or several messages stuck together. So you must loop until exactly the needed number of bytes has been received (see `recv_exact` in Part 2).

13. Why is file metadata sent before the actual file?

The receiver must know *what* is coming (filename) and *how many bytes* (filesize) before reading. Otherwise it cannot name the file, cannot tell where it ends, and cannot separate raw file bytes from the next JSON message.

14. Why are files transferred in chunks?

To avoid loading the whole file into RAM. A 2 GB video read at once could crash the program. With 64 KB chunks memory use stays small and constant, and progress can be reported.

15. How does the receiver know the file transfer has finished?

It counts bytes: it keeps reading until `received == filesize` from the metadata. It does not depend on one `recv()` call or on the connection closing.

16. What happens if the other peer disconnects?

`recv()` returns empty bytes (clean close) or raises `ConnectionResetError / BrokenPipeError`. The program catches it, closes that socket, removes the peer from the list, logs a message and keeps running. It must not crash.

17. Where is the received file stored?

In the local `downloads/` folder, e.g. `downloads/photo.jpg`.

18. Is there a central server in this system?

No. Peers connect directly to each other. Each peer has its own listening socket; the only "server" is each peer's own server role.

19. What if the central server in a client-server system went offline?

It is a **single point of failure**: no client can communicate and the service stops. In P2P, if one peer goes down the others can still talk to each other.

20. What challenges exist for a large-scale Internet P2P system?

- **Peer discovery**: finding peers without a central list (DHT, bootstrap nodes, gossip).
- **NAT / firewall traversal**: peers behind routers lack public IPs (STUN, TURN, hole punching).
- **Security**: encryption, authentication, malicious peers, fake identities.
- **Scalability and routing**: efficient lookup across millions of nodes.
- **Churn**: peers constantly joining and leaving.
- **Data integrity and availability**: file hashing, replication, chunk recovery.
- **Consensus and trust**: needed in systems such as blockchain.
- **Fairness**: free-riders that download but never upload.

Part 2: Core Concepts for the Viva

Client-Server vs P2P

	Client-Server	P2P
Structure	Central server	Equal peers
Failure	Single point of failure	Resilient
Scalability	Server is a bottleneck	Scales with number of peers
Cost	Server infrastructure	Shared among peers
Control	Centralized	Decentralized
Examples	Web, email	BitTorrent, blockchain

Socket Lifecycle

```
Server: socket() -> bind() -> listen() -> accept() -> recv()/send() -> close()
Client: socket() -> connect() -> send()/recv() -> close()
```

- **bind()** attaches the socket to an IP:port. **listen()** marks it passive and queues incoming connections.
- **0.0.0.0** listens on all interfaces (needed for LAN access); **127.0.0.1** is loopback only (same machine).

TCP Key Points

- Connection-oriented, reliable, ordered, byte-stream protocol.
- Three-way handshake: SYN, SYN-ACK, ACK. Includes flow control and congestion control.
- `sendall()` guarantees all bytes are sent; `send()` may send only part.

Message Framing (Length Prefix)

- `struct.pack("!I", n)`: **!** = network byte order (big-endian), **I** = 4-byte unsigned int. Max message size is about 4 GB.
- Receiver reads 4 bytes, decodes length N, then reads exactly N bytes.

```
import struct, json

def recv_exact(sock, n):
    data = b""
    while len(data) < n:
        chunk = sock.recv(n - len(data))
        if not chunk:
            raise ConnectionError("Peer disconnected")
        data += chunk
    return data

def send_msg(sock, obj):
    data = json.dumps(obj).encode()
    sock.sendall(struct.pack("!I", len(data)) + data)

def recv_msg(sock):
    length = struct.unpack("!I", recv_exact(sock, 4))[0]
    return json.loads(recv_exact(sock, length).decode())
```

File Transfer Flow

- **Sender:** send JSON metadata {type:file, filename, filesize}, then read the file in 64 KB chunks and `sendall(chunk)`.

- **Receiver:** parse metadata, then loop `recv(min(64KB, remaining))` and write to `downloads/filename` until remaining is 0.
- **Binary mode (rb/wb):** a file is just bytes, so one protocol handles images, audio, video, PDF, ZIP. Text mode would corrupt binary data.
- **Subtle point:** reading exactly `min(chunk, remaining)` ensures you never consume bytes of the next JSON message.

```
# Receiver loop
remaining = filesize
with open(path, "wb") as f:
    while remaining > 0:
        chunk = sock.recv(min(64 * 1024, remaining))
        if not chunk:
            raise ConnectionError("Peer disconnected mid-file")
        f.write(chunk)
        remaining -= len(chunk)
```

Threads and Concurrency

- Blocking calls: `accept()`, `recv()`, `connect()`.
- One thread accepts connections; one thread per peer connection. Use `daemon=True` so threads stop on exit.
- Protect shared data (the peers dictionary) with `threading.Lock()` to avoid race conditions.
- Tkinter is not thread-safe: update the GUI from network threads via `root.after()` or a queue.

Peer Identity

- **peer_id:** unique random ID (e.g. `uuid.uuid4().hex[:8]`) because names can repeat.
- HELLO carries the peer's **listening port**. The port seen on an accepted connection is a random **ephemeral port** and cannot be used to reconnect.

Error Handling

Situation	How to handle
Invalid IP / port	Validate input; catch <code>ValueError</code> / <code>socket.gaierror</code>
Peer not running	Catch <code>ConnectionRefusedError</code> , show [ERROR] message
Slow / unreachable host	Use <code>settimeout()</code> on connect; catch <code>socket.timeout</code>
Peer disconnects	Handle empty <code>recv</code> , <code>ConnectionResetError</code> , <code>BrokenPipeError</code> ; remove peer
File missing / invalid size	Check <code>os.path.exists()</code> ; validate filesize before sending
No peer selected	Show a warning, do not send
Thread cleanup	try / except / finally and always close the socket in finally

Part 3: Likely Follow-up Questions

Two peers use the same port on one machine?

Error: `OSError: Address already in use.`

What is `SO_REUSEADDR`?

Lets a server restart immediately without waiting for the old socket's `TIME_WAIT` to expire.

What is an ephemeral port?

A temporary port the OS assigns to the client side of a connection.

Why 0.0.0.0 instead of 127.0.0.1 for the final demo?

127.0.0.1 accepts only local connections; 0.0.0.0 accepts connections from other LAN machines.

Why might a LAN connection fail?

Firewall blocks the port, computers are on different networks, or the IP is wrong. Check with `ipconfig / ifconfig`.

Why JSON for messages?

Human-readable, easy to parse, language-independent. Downside: size overhead and it cannot carry raw binary, so file bytes are sent separately.

Is the communication secure?

No. Plain text, no encryption or authentication (out of scope). Anyone sniffing the network could read it. TLS would fix this.

Can two peers connect to each other twice?

Possibly, unless you check `peer_id` and reject duplicates.

TCP or UDP for a video call?

UDP, because low latency matters more than perfect delivery.

What is NAT and why is it a P2P problem?

Home routers share one public IP among many devices and block incoming connections to private IPs by default, so peers cannot reach each other directly.

Is this fully decentralized?

Yes for communication, but peers must still know each other's IP:port manually (no discovery).

How does this relate to blockchain?

Blockchain nodes are peers that connect directly, broadcast transactions/blocks to neighbours and keep copies of the ledger. This assignment is the networking foundation beneath that.

Part 4: The Full Communication Chain (Section 21)

Stage	Explain in one line
Peer A	The sending application / user.
IP address + port	Locates the machine and the application on it.
TCP socket	Reliable byte-stream connection established via three-way handshake.
HELLO handshake	Peers exchange identity (ID, name, listening port).
Application protocol	Length-prefixed JSON messages with a type field: hello, hello_ack, text, file.
Text / file data	Text as JSON; files as metadata followed by raw chunked bytes.
Peer B	Receiver parses, displays text or saves the file to downloads/.

Viva Tips

- Know your own code line by line, especially `recv_exact`, the framing and the file-receive loop.

- Be able to draw the architecture on paper: User Interface, P2P Node, TCP Socket, Other Peer.
- Explain *why*, not just *what*. The viva carries the largest share of marks, and a simple implementation you can explain beats a complicated one you cannot.